

MATRICES CREUSES

Dans ce problème, nous cherchons à effectuer la multiplication d'une matrice creuse A de taille « logique » ($N \times N$) par un vecteur X de taille N, le résultat étant stocké dans un vecteur Y de taille N. Nmax désignera le nombre maximum d'éléments non nul par ligne et Nnz, le nombre total d'éléments non nuls de A. Nous supposerons qu'il y a au moins un élément non nul par ligne.

Dans la première partie de ce problème, nous utiliserons un stockage de type CSR (Compressed Storage Row) défini de la manière suivante :

- A_VAL(N, Nmax) est un tableau bi dimensionnel de flottants dont la ligne I contient les éléments non nuls de la ligne I de la matrice « logique » A, éventuellement cette ligne sera complétée par des 0.
- A_IND(N, Nmax) est un tableau bi dimensionnel dont la ligne I contient les numéros de colonne des éléments non nuls de la matrice « logique » A. A_IND(I,K) contient le numéro de colonne de l'élément de matrice non nul dont la valeur est stockée dans A_Val(I,K).
 1. Ecrire l'algorithme correspondant pour effectuer l'opération de produit matrice vecteur utilisant le stockage CSR décrit ci-dessus
 2. Evaluer le nombre d'opérations flottantes et aussi le nombre de tests de l'algorithme proposé
 3. Rajouter un tableau NNZ(N) tel que NNZ(I) contienne le nombre d'éléments non nuls de la ligne I de la matrice A. Utiliser ce tableau pour concevoir un nouvel algorithme permettant de réduire le nombre d'opérations flottantes.

1. On cherche à calculer $Y = AX$ $Y_i = \sum_{j=1}^N a_{ij} x_j$ où $A \in \mathcal{M}_{n \times n}(\mathbb{R})$ et A est stockée sous une forme de type CSR à ligne de longueur fixe.

A_VAL[i,k] : la valeur de k-ième éléments non null de i-ième ligne.

A_IND[i,k] : l'indice de colonne correspondant de A_VAL[i,k].

Algorithme est alors :

DO I = 1, N

DO K = 1, Nmax

IF A_IND[I,K] != 0 DO

J = A_IND[I,K]

Y[I] = Y[I] + A_VAL[I,K] * X[J]

ENDIF

ENDDO

ENDDO

2. Dans le pire des cas, chaque ligne contient Nmax éléments non nuls.

Pour chaque élément, on effectue une addition et une multiplication.

Donc le nombre total de FLOPs est $2 \times N \times N_{max} = O(N \times N_{max})$

Et pour chaque élément stocké, on effectue un test $A_IND[I,K] != 0$,

donc le nombre total de tests est $N \times N_{max} = O(N \times N_{max})$

3. $NNZ[I]$ est nombre d'éléments non nuls de i -ième ligne.

Algorithme est alors

DO $I = 1, N$

DO $K = 1, NNZ[I]$

$J = A_IND[I, K]$

$Y[I] = Y[I] + A_VAL[I, K] * X[J]$

ENDDO

ENDDO

Comme $NNZ[I]$ donne le nombre exact d'éléments non nuls de la ligne i , aucun test conditionnel n'est nécessaire.

Dans la deuxième partie de ce problème, nous utiliserons un stockage de type CSR Avancé dans lequel seuls les éléments non nuls sont stockés. Ce stockage est défini de la manière suivante :

- $A_VAL_ADV(Nnz)$ est un tableau unidimensionnel de flottants qui contient tous les éléments non nuls de A stockés ligne par ligne et par numéro de ligne croissant.
 - $A_IND_ADV(Nnz)$ est un tableau unidimensionnel d'entiers qui contient tous les numéros de colonne des éléments non nuls de A stockés ligne par ligne et par numéro de ligne croissant. $A_IND_ADV(K)$ contient le numéro de colonne de l'élément non nul de la matrice dont la valeur est stockée dans $A_VAL_ADV(K)$
 - $Beg_Row(N)$ est un tableau unidimensionnel d'entiers qui contient les numéros d'indice dans les tableaux A_VAL_ADV et A_IND_ADV , des débuts de ligne. $Beg_Row(I)$ contient l'indice où démarre le stockage des informations relatives à la ligne I
 - $End_Row(N)$ est un tableau unidimensionnel d'entiers qui contient les numéros d'indice dans les tableaux A_VAL_ADV et A_IND_ADV , des fins de ligne. $End_Row(I)$ contient l'indice où finit le stockage des informations relatives à la ligne I
4. Ecrire l'algorithme correspondant pour effectuer l'opération de produit matrice vecteur utilisant le stockage CSR décrit ci-dessus
 5. Evaluer le nombre d'opérations flottantes et aussi le nombre de tests de l'algorithme proposé
 6. Comparer par rapport à l'algorithme précédent en terme de stockage et de nombre d'opérations (flottantes et branchements)
 7. Montrer comment les tableaux $Beg_Row(N)$ et $End_Row(N)$ peuvent être fusionnés en un seul et même tableau

4. D'après les définitions dans l'énoncé, on a alors Algorithme :

DO $I = 1, N$

DO $K = Beg_Row[I], End_Row[I]$

$J = A_IND_ADV[K]$

$Y[I] = Y[I] + A_VAL_ADV[K] * X[J]$

ENDDO

ENDDO

5. Pour chaque élément non nul, on effectue une addition et une multiplication.

Donc le nombre de FLOPs est donc $2 * Nnz = O(Nnz)$

Et 0 test conditionnel.

6. Dans le stockage CSR simple, les tableaux A-VAL et A-IND sont de taille $N \times N_{\max}$, donc la taille totale est $2NN_{\max}$.

Et pour le stockage CSR avancé, les tableaux A-VAL-ADV et A-IND-ADV sont de taille N_{nz} , et les tableaux Beg-Row et End-Row sont N , alors la taille totale est $2N + 2N_{nz}$.

D'après les questions précédentes, le nombre d'opération (FLOPs et test) de CSR simple est $3NN_{\max}$, celle de CSR avancé est $2NN_{nz}$

7 Format 1) :

Stocker comme une liste nombre d'élément pour chaque ligne.

Nombre d'opération $\uparrow$ (calcul indice)

Format 2) :

Stocker comme $[\text{Beg-Row}(1), \text{End-Row}(1), \dots, \text{Beg-Row}(N), \text{End-Row}(N)]$

Stockage $\uparrow$

Format 3) :

Fusionner en un seul tableau Row-Ptr défini par :

$$\begin{cases} \text{Row-Ptr}[I] = \text{Beg-Row}[I] & \forall I \in \{1, \dots, N\} \\ \text{Row-Ptr}[N+1] = \text{End-Row}[N] + 1. \end{cases}$$

Alors les éléments non nuls de la ligne I sont alors stockés entre $\text{Row-Ptr}[I]$ et $\text{Row-Ptr}[I+1] - 1$.